

RESPUESTAS PRUEBA TÉCNICA ESCRITA

DESARROLLADOR ODOO (v19)

Parte A — Análisis y planificación (sin código)

1. modelo de datos:

Modelo: library.rental.category

Campos principales:

- name tipo Char
- active tipo Boolean
- max_simultaneous_loans tipo Integer

Justificación: Se crea el modelo por estandarización del código. Aunque se podría heredar del modelo product.category considero que es mejor opción separar los modelos para aislar la lógica de ventas y productos.

Modelo: library.rental.item

Campos principales:

- name tipo Char
- default_code tipo Char
- category_id tipo Many2one relacionado al modelo library.rental.category
- state tipo Selection con estas opciones (disponible, prestado, en_reparacion, perdido)
- active tipo Boolean
- loan_ids tipo One2many con relacion al modelo library.rental.loan y el campo item_id

Justificación: Se crea el modelo por estandarización del código. Aunque se podría heredar del modelo de producto considero que es mejor crear este modelo y aislar la lógica transaccional, evitando el overhead de los cálculos de inventario nativos (como valoraciones y stock a mano), lo que garantiza consultas rápidas y escalables

Modelo: library.rental.loan

Campos principales:

- item_id tipo Many2one con relación al modelo library.rental.item
- employee_id tipo Many2one con relación al modelo de hr.employee
- date_start tipo Date
- date_due tipo Date
- state tipo Selection con estas opciones (activo, devuelto, vencido, perdido)

Justificación: Se crea el modelo por que no hay un modelo que nos permita tener un historial de préstamos así que se implementa para llevar un mejor control sobre al empleado, el ítem y las fechas.

2. Seguridad:

- Grupos (res.groups):
 - library_rental.group_employee (Usuario / Empleado): Acceso básico.
 - library_rental.group_librarian (Manager / Bibliotecario): Acceso total.
- Permisos en ir.model.access.csv:
 - Empleado: Solo lectura (perm_read=1) en categorías, ítems y préstamos.
 - Bibliotecario: Permisos totales (1,1,1,1) en los tres modelos.
- Reglas de registro (ir.rule) para "ver sólo su propio historial":
 - Se requiere una regla sobre el modelo library_rental.loan vinculada al grupo group_employee.
 - Dominio: [('employee_id.user_id', '=', user.id)]. Esto garantiza que un usuario de Odoo solo pueda ver los registros de préstamo donde el empleado asociado coincida con su usuario logueado.
 - Para el bibliotecario, se crea otra regla [(1, '=', 1)] para que vea todo.

3. Rendimiento a escala:

- indexación de la BD: se debe agregar el atributo "index=True" a campos clave para las búsquedas y agrupación. La recomendación sería aplicar el atributo en los siguientes campos:
 - Modelo library_rental.item: Campos state y category_id
 - Modelo library_rental.loan: date_due y state
- En caso de que la BD crezca rápidamente y tenga demasiados registros se podría crear una vista SQL para realizar los reportes de forma más eficiente.

4. Automatización:

Para la detección de vencidos se deben usar dos mecanismos:

- Cron o Acción planificada que se ejecuta todos los días en un horario en que no haya mucha afluencia de usuarios. Este proceso realizaría una búsqueda en el modelo library_rental.loan para filtrarían todos los registros en donde el date_due sea menor a la fecha actual y el estado state activo con el resultado de la búsqueda se procede a realizar el proceso de notificación y cambio de estado.
- Para las Notificaciones se implementa de dos formas:
 - Enviar un correo o mensaje al empleado informando que su préstamo está vencido.
 - Se crearía una actividad planificada al bibliotecario para que revise los registros y realice el debido proceso.

5. Riesgos y supuestos:

Supuesto 1: Licenciamiento y acceso de los empleados (Usuarios Internos)

- El supuesto: Para cumplir el requerimiento de "un empleado ve solo su propio historial" mediante reglas de registro (ir.rule en el backend), asumo que los 1.200 empleados cuentan con un usuario interno activo (res.usersvinculado a base.group_user).
- El riesgo: En el sector retail, es altamente improbable que el 100% del personal de piso o bodega tenga licencias de usuario interno por una cuestión de costos.
- Mitigación: Si la mayoría no tiene usuario interno, la solución no puede depender del backend. El diseño debe pivotar hacia la creación de un grupo de Usuarios de Portal (gratuitos) y el desarrollo de un controlador web (/my/loans) heredando de portal.CustomerPortal, de modo que el empleado acceda a su historial desde la web sin consumir licencia ERP.

Supuesto 2: Dependencia arquitectónica del módulo de Recursos Humanos (hr)

- El supuesto: Al utilizar el modelo hr.employee para relacionar los préstamos con las personas, asumo que el cliente ya tiene instalado o planea implementar el ecosistema de Recursos Humanos nativo de Odoo.
- El riesgo: Si el cliente gestiona su nómina y personal en un software de terceros y se niega a instalar el módulo hr, nuestro módulo fallará por dependencias huérfanas.
- Mitigación: Si se confirma este escenario, se modificaría el modelo de datos para apuntar al modelo res.partner (Contactos), agregando un campo is_employee = fields.Boolean() o utilizando Categorías de Contacto (Tags) para identificar a los empleados, manteniendo el módulo de biblioteca totalmente independiente del módulo de Recursos Humanos.

2. Parte B — Seguridad (revisión de código)

Código corregido con comentarios de los cambios aplicados:

```
from odoo import http
from odoo.http import request
from odoo.exceptions import AccessError, UserError

class PartnerOrderController(http.Controller):

    @http.route('/api/order/status', type='json', auth='user')
    def get_order_status(self, order_id):
        # Solución: Uso del ORM (seguro contra inyección SQL y aplica reglas de acceso nativas). El
        # SQL directo va en contra de los estándares de Odoo. Solo se debe usar cuando no se pueda
        # completar la consulta a través del ORM
        # Auth='user' garantiza que quien consulta tiene sesión activa.
        order = request.env['sale.order'].search([('id', '=', int(order_id))], limit=1)
        if not order:
            return {'error': 'not found or access denied'}
        return {'id': order.id, 'state': order.state, 'amount_total': order.amount_total}

    @http.route('/api/order/update_state', type='json', auth='user')
    def update_order_state(self, order_id, new_state):
        # Solución: Eliminar sudo(). Validar cambios de estado mediante métodos de negocio.
        order = request.env['sale.order'].browse(int(order_id))
        if not order.exists():
            return {'error': 'not found'}
        # En Odoo, el estado no se cambia directamente ya que hay métodos definidos para cada caso
        # donde se corren procesos internos para validar la integridad de la BD según el estado.
        try:
            if new_state == 'sale':
                order.action_confirm()
            elif new_state == 'cancel':
                order._action_cancel()
            else:
                return {'error': 'invalid state transition'}
        except (UserError, AccessError) as e:
            return {'error': str(e)}
        return {'result': 'ok'}

    @http.route('/api/order/list', type='json', auth='user')
    def list_orders(self):
        # Solución: Eliminar sudo() y el parámetro libre partner_id.
        # El ORM filtra automáticamente por el usuario logueado y las reglas de acceso.
        partner_id = request.env.user.partner_id.id
        orders = request.env['sale.order'].search([('partner_id', '=', partner_id)])
        return {'orders': [{'id': o.id, 'amount_total': o.amount_total} for o in orders]}
```

Respuestas a Preguntas:

1. Esto sería un caso clásico de inyección SQL en el cual se podría obtener todas los registros saltándose las reglas de acceso.
Las correcciones requeridas son:
 - Por estándares de Odoo no se deben usar consultas SQL directas si existe un equivalente en el ORM; el uso del ORM garantiza que se apliquen las reglas de acceso nativas.
 - En caso de que se deba usar SQL directo, los parámetros de consulta siempre se deben pasar a través de una tupla en el `cr.execute()` para neutralizar la inyección.
2. Esta combinación se podría aplicar en integraciones de webhooks de pasarelas de pago como MercadoPago. En esos casos el payload llega con una firma criptográfica y el controlador debe validar la firma antes de realizar algún cambio por motivos de seguridad. El cambio propuesto sería cambiar a `auth='user'` (si el usuario opera desde el navegador con sesión) o implementar una validación por API Key / Token de acceso para garantizar la autenticidad de la petición externa.
3. Para este caso el problema es que la consulta se está realizando con `sudo`, lo cual eleva los permisos, anula las reglas de seguridad del sistema y crea una vulnerabilidad. La solución es eliminar el `sudo()`, obtener el `partner_id` directamente del usuario logueado en la sesión y realizar la búsqueda normal mediante el ORM, garantizando que el usuario solo pueda ver sus registros asociados.
4. Al cambiar solamente el estado a la orden se rompen varios procesos internos. Por ejemplo al pasar la orden de borrador a confirmada se generan los albaranes de entrega que a su vez generan movimientos de inventario; de igual forma, si se cancela la orden, se debe liberar el inventario que ya estaba reservado. Cambiar el estado a la fuerza altera la vista, pero no ejecuta esta lógica subyacente, generando inconsistencias de datos.
5. Al eliminar el uso del `sudo()`, el ORM de Odoo tomará automáticamente el usuario de la sesión actual y registrará el cambio en el *chatter* del modelo, aprovechando el *tracking* nativo heredado de `mail.thread`. En caso de que sea necesario usar el `sudo` se debe implementar el uso del metodo `.with_user(user_id)` antes de modificar el registro para que odoo realice el cambio con el usuario real y así quede registrado correctamente el cambio en el chatter.

3. Parte C — Rendimiento (revisión de código)

Código ajustado y comentado:

```
def _cron_update_partner_risk_score(self):
    # 1. Obtener todos los partners de interés
    partners = self.env['res.partner'].search([('customer_rank', '>', 0)])
    if not partners:
        return

    # 2. Agrupación 1: Total facturado por partner (evita cargar facturas en RAM)
    # _read_group en Odoo 19 devuelve un iterador de tuplas.
    invoice_groups = self.env['account.move']._read_group(
        domain=[
            ('partner_id', 'in', partners.ids),
            ('move_type', '=', 'out_invoice')
        ],
        groupby=['partner_id'],
        aggregates=['amount_total:sum']
    )
    invoiced_by_partner = {partner.id: total for partner, total in invoice_groups}

    # 3. Agrupación 2: Conteo de facturas vencidas por partner
    late_groups = self.env['account.move']._read_group(
        domain=[
            ('partner_id', 'in', partners.ids),
            ('move_type', '=', 'out_invoice'),
            ('invoice_date_due', '!=', False),
            ('payment_state', '!=', 'paid')
        ],
        groupby=['partner_id'],
        aggregates=['__count']
    )
    late_by_partner = {partner.id: count for partner, count in late_groups}

    # 4. Escribir los resultados iterando los partners en memoria
    # Pasamos de O(N*M) consultas a solo O(N) writes, eliminando todas las
    # búsquedas.
    for partner in partners:
        total_amount = invoiced_by_partner.get(partner.id, 0.0)
        total_late = late_by_partner.get(partner.id, 0)

        score = 'high_risk' if total_late > 3 else 'low_risk'

        # Se omite el commit manual. El framework confirma todo de forma segura al final.
        partner.write({
            'risk_score': score,
            'total_invoiced': total_amount,
        })
```

Respuestas a Preguntas:

1. Cálculo paso a paso:

- Se lanza **1** consulta para obtener los N partners.
- Por cada partner (N), se lanza **1** consulta para buscar sus órdenes.
- Por cada orden (M) de ese partner, se lanza **1** consulta para buscar facturas.
- Por cada partner, se lanza **1** consulta para hacer el write.
- El total de consultas es: $1+N+(N \times M)+N = 1+2N+(N \times M)$. Con $N=80.000$ y $M=6.25$ ($500.000 / 80.000$), el método dispara **660.001 consultas SQL**. Escala así (de forma multiplicativa) porque se está iterando y consultando a la base de datos dentro de ciclos anidados, en lugar de utilizar operaciones de agregación y conjuntos.

2. Llamar a `commit()` en cada ciclo hace que la BD se sature. Si el cron se cae por un *timeout* en el registro 40.000, los primeros 39.999 quedarán confirmados de forma irreversible, dejando la base de datos en un estado inconsistente. Como la búsqueda inicial no filtra a los ya procesados, el siguiente intento del cron volverá a empezar desde cero, pudiendo no terminar nunca.
3. agregaría un índice compuesto (B-Tree) sobre la tabla `account_move` utilizando los campos (`partner_id`, `move_type`, `payment_state`). Como Odoo ya indexa `partner_id` por defecto, este índice compuesto permitiría que PostgreSQL resuelva el GROUP BY y los filtros de la segunda agrupación de forma casi instantánea directamente desde el índice.
4. Asume erróneamente que la relación es 1 a 1 y que el nombre nunca cambia. Si el cliente unifica dos pedidos en una sola factura, el campo `invoice_origin` guardará un texto separado por comas (ej. "SO001, SO002"). Al usar el operador de igualdad (`=`), la búsqueda fallará y omitirá esa factura.
La forma estándar y nativa de Odoo de enlazar esto es utilizar el campo relacional Many2many **`invoice_ids`** presente en el modelo `sale.order` (o navegar por las líneas ya que mantienen la integridad referencial a nivel de base de datos sin depender de textos).

4. Parte D — Preguntas puntuales sobre Odoo 19

Respuestas a Preguntas:

1. En Odoo 19, usar `_sql_constraints` está prohibido; el sistema solo emite un WARNING y **la restricción no se crea en la base de datos**. Migrar a `models.Constraint()` es obligatorio para que la regla se aplique realmente a nivel de PostgreSQL.

2. `read_group()` quedó obsoleto por ineficiencia. La diferencia clave al migrar es que el antiguo método devuelve una lista de diccionarios, mientras que `_read_group()` devuelve directamente un iterador de tuplas puras, lo que requiere desempaquetar los resultados en el código Python.
3. Usar los atributos privados rompe la propagación correcta del contexto del entorno. Además, si el framework modifica su implementación interna de la caché de sesión o transacciones, el código fallará. Siempre se debe interactuar a través de la API pública (`self.env`).
4. En Odoo 19, el core ha migrado a módulos nativos de ECMAScript (ES modules), por lo que el comentario de transpilación `@odoo-module` ya no se usa. Un candidato solo lo agregaría si está escribiendo código *legacy* que necesita convivir simultáneamente con módulos antiguos en un entorno de transición.

5. Parte E — OWL (frontend)

Código ajustado y comentado:

```
# JavaScript

import { Component, useState, onWillStart } from "@odoo/owl";
import { useService } from "@web/core/utils/hooks";

export class RentalDashboard extends Component {
  static template = "library_rental.RentalDashboard";
  // Mantiene la validación estricta de props
  static props = {};

  setup() {
    this.orm = useService("orm");
    this.notification = useService("notification");

    // Corrección 1: Envolver el estado con useState para reactividad
    this.state = useState({ items: [], loading: true });

    // Corrección 2: Usar el hook de ciclo de vida correcto para cargas iniciales
    onWillStart(async () => {
      await this.loadItems();
    });
  }

  async loadItems() {
    this.state.loading = true;
    try {
      this.state.items = await this.orm.searchRead(
        "library.item", [], ["name", "state"]
      );
    }
  }
}
```

```

    } catch (error) {
      // Corrección 3: Manejo de errores con el servicio de notificación
      this.notification.add("Error al cargar los ítems", { type: "danger" });
    } finally {
      this.state.loading = false;
    }
  }
}

onClickRefresh() {
  this.loadItems();
}
}

# XML
<t t-name="library_rental.RentalDashboard">
  <div class="o_rental_dashboard">
    <button t-on-click="onClickRefresh">Refrescar</button>
    <t t-if="state.loading">
      <div>Cargando...</div>
    </t>
    <t t-else="">
      <!-- Corrección 4: Uso de t-key obligatorio en t-foreach -->
      <t t-foreach="state.items" t-as="item" t-key="item.id">
        <div t-esc="item.name"/>
      </t>
    </t>
  </div>
</t>

```

Respuestas a Preguntas:

1. Al usar un objeto plano en lugar de `useState()`, el estado pierde su reactividad. Aunque el método `loadItems()` modifique el array internamente tras presionar el botón "Refrescar", OWL no detecta la mutación y no dispara el renderizado, dejando la interfaz congelada.
2. El lugar correcto para consultas iniciales es el hook `onWillStart()`. Llamar a `loadItems()` directamente en `setup()` sin un `await` seguro provoca que la promesa se resuelva en segundo plano. Si el componente se desmonta antes de que la promesa termine, el código intentará actualizar el estado de un componente destruido, generando un error en consola y fugas de memoria.
3. Sin un bloque `try/catch`, si la consulta falla (ej. `AccessError`), el usuario sufrirá un error silencioso: la interfaz quedará bloqueada en estado de carga infinita. Para mostrar el error de forma consistente, se debe inyectar el servicio nativo de Odoo mediante `useService("notification")` y mostrar un toast de alerta.
4. Sin `t-key`, el DOM virtual de OWL no puede identificar unívocamente cada elemento de la lista y se guía por el índice de posición. Si el orden de los

ítems cambia en un refresco, OWL reutilizará nodos DOM incorrectos, lo que causa parpadeos visuales o que componentes hijos retengan datos de otro registro.

5. Es una buena práctica porque activa la validación estricta de propiedades del framework. Si otro desarrollador pasa un prop por accidente y no existe la declaración, OWL lo aceptará silenciosamente, lo que puede esconder bugs difíciles de rastrear. Con `static props = {}`, OWL lanzará un error crítico y explícito en la consola inmediatamente al detectar el prop no deseado.